Multi-Model Adapters for Agentic Skills

https://gemini.google.com/app/a0be6c1882046bb3

User prompt: topic is: creation of multi-model adapters for agentic plugins and skills

Response: Let's explore the creation of multi-model adapters for agentic plugins and skills together; I'll ask guiding questions along the way.

As AI agents 🤖 evolve from isolated scripts into complex orchestrations, they require composable methods to acquire domain-specific expertise. These capabilities are often packaged as skills 🤹—directories containing instructions, scripts, and resources. However, different agent harnesses (like Claude Code or Cursor) and underlying models have unique ways of processing context. A multi-model adapter 🔌 acts as a translation layer. Instead of reducing a skill to a basic text prompt, the adapter natively structures the plugin for each specific environment, managing interoperability protocols 🤝, context token optimization 🪙, and tool bindings 🛠️.

Here are three pathways we can take to break this down. Which of these areas would you like to focus on first?

1. 🤹 **Skill Architecture**: How to structure directories and package instructions so they can be dynamically discovered and loaded by an agent.
2. 🔌 **Adapter Mechanics**: Designing the translation layer that converts a universal skill format into harness-native artifacts and handles communication protocols.
3. 🪙 **Token Optimization**: Strategies for progressive disclosure and memory persistence when feeding complex skills into multi-model workflows.

Sources

1. Equipping agents for the real world with Agent Skills - Anthropic

---

User prompt: lets do 1, 2 and 3 in order

Response: Let's start with the foundation: **Skill Architecture** 🏗️.

For an agent to dynamically load a new capability, the skill cannot just be a loose text file. It requires a predictable contract—a self-contained package that an adapter can confidently parse, regardless of the ultimate model executing it.

To ensure a single source of truth, we can structure this alongside authoritative documentation. A scalable pattern within a workspace like `aplicatudo-monorepo` involves isolating skills into a dedicated registry, perhaps within the existing `AI_Codex` directory:

```
aplicatudo-monorepo/
└── AI_Codex/
    └── Skills/
        └── DartRefactor/
            ├── manifest.json       (The Contract)
            ├── instructions.md     (The Core Logic)
            └── tools/
                └── ast_parser.sh   (Executable Actions)
```

The Three Pillars of a Skill Package

1. 📜 **The Manifest (`manifest.json`):** This is the machine-readable metadata. It defines the skill's name, version, intended use cases, and the schema for any required input variables (e.g., specifying that it expects a `target_directory` string).
2. 🧠 **The Instructions (`instructions.md`):** The human-readable (and LLM-readable) directives. This contains the persona, the step-by-step reasoning framework, and the specific constraints the agent must follow.
3. 🛠️ **The Tools (`/tools`):** The isolated scripts or binaries the agent can invoke to interact with the environment. By keeping these as CLI tools rather than tightly coupled IDE plugins, they remain portable across different agent harnesses.

This architecture intentionally separates the *what* (instructions) from the *how* (tools) and the *rules* (manifest).

Before we move on to step 2—how the **Adapter Mechanics** 🔌 actually translate this package—consider the `instructions.md` file. Different models handle context differently; some rely heavily on a distinct `system` prompt, while others expect instructions interwoven into the `user` message history.

How should our adapter approach parsing this single `instructions.md` file so that it remains effective across models with completely different prompting paradigms?

User prompt: it should implement a middleware to deliver the instructions in the format and style each model expects

Response: Exactly. The adapter acts as a middleware pipeline, and at its core, it is essentially a series of pure transformations. It takes the universal skill package (manifest, instructions, and tools) and pipes it through model-specific formatters without mutating the original source of truth in `AI_Codex`.

Here is how that middleware architecture breaks down into distinct translation layers.

1. Context Shaping (The Dialect Translator)

Different models have distinct "dialects" that trigger their best performance. If we feed raw Markdown to a model heavily fine-tuned on XML, we lose adherence to the instructions. The middleware maps the universal `instructions.md` to the target model's preferred structural paradigm.

Target Model	Preferred Paradigm	Middleware Transformation Strategy
Claude (Anthropic)	XML Tagging & Document structure	Wraps instructions in explicit `<system>`, `<persona>`, and `<constraints>` XML tags. Places tools inside `<tools>` schemas.
GPT-4 (OpenAI)	System Prompt Dominance	Compresses instructions into a dense, high-priority `system` role message. Prefers JSON-like structured constraints over XML.
Gemini (Google)	Markdown & Clear Hierarchy	Retains structured Markdown (headings, bullet points). Prefers instructions grouped by logical sections rather than heavy markup.

By treating the parsed `instructions.md` as an immutable input state, the middleware can use a factory pattern (or a switch on the model type) to route the data through the appropriate pure function (e.g., `toAnthropicDialect(instructions)`, `toOpenAiDialect(instructions)`), returning the formatted payload.

2. Tool Binding Translation

The tools located in the skill's `/tools` directory also require translation. An underlying CLI script (like an AST parser) doesn't change, but the way the model *knows* how to call it does.

The middleware reads the `manifest.json` and maps the tool schemas into the specific JSON format expected by the model's API:

- **OpenAI:** Translates to the `functions` array within the `tools` parameter.
- **Claude:** Translates to Anthropic's specific tool-use schema.
- **CLI Harness:** If operating strictly via a terminal interface, the middleware might format the tools as distinct command-line execution instructions within the prompt itself.

3. Protocol Management

Finally, the middleware normalizes the output. When the model decides to invoke a tool, it might reply with a JSON block, a custom tool-call token, or an XML tag. The middleware catches this output, parses it back into a universal command, executes the isolated script, and feeds the deterministic result back into the model's context window.

This functional translation pipeline ensures that your skills remain entirely decoupled from the models executing them.

As these interactions loop, the context window fills up quickly with tool outputs and instruction history. This leads us directly to the third topic: **Token Optimization**.

When a skill requires a long-running, multi-step execution (like traversing a codebase), how would you prefer to handle the memory state so we don't blow past the model's context limit?

User prompt: spawn specialized sub-agents to handle sub-tasks, with the main agent acting as an orchestrator. that way, the main context is preserved, as sub-agents have their own context window. as a bonus, since sub-tasks are more focused, the sub-agents's context will consist only of information that is relevant to the task.

Response: That is a highly effective architecture 🎯. By shifting from a single monolithic agent to an **Orchestrator-Worker pattern**, we naturally solve the context bloat problem.

In this design, the Orchestrator agent maintains the high-level goal, state ledger, and routing logic, while specialized sub-agents are spawned on demand to execute isolated sub-tasks (e.g., writing a single test file or parsing an AST). Once a sub-agent finishes, it returns only its deterministic final artifact or a concise summary to the Orchestrator, and its local context window is completely discarded 🗑️. This process keeps the Orchestrator's context remarkably clean and focused.

The Math Behind Context Isolation

If we execute N sub-tasks sequentially in a single monolithic context window, where each task generates T tokens of conversation and tool logs, the total context grows linearly or quadratically depending on history retention:

$$\text{Monolithic Context} = \sum_{i=1}^{N} T_i$$

With an Orchestrator-Worker setup, the Orchestrator only absorbs a tiny summary (S) of each task:

$$\text{Orchestrator Context} = \sum_{i=1}^{N} S_i \quad \text{where } S_i \ll T_i$$

To help visualize how this distribution impacts token efficiency and prevents context overflow, let's explore this interactive simulator. You can adjust the task complexity and see exactly how the Orchestrator shields its context compared to a traditional monolithic agent.

Now that we have covered **Skill Architecture** (1), **Adapter Mechanics** (2), and **Token Optimization via Orchestration** (3), we have a complete blueprint for portable, efficient agentic skills.

When the Orchestrator receives a highly complex task that spans multiple disciplines, how should it handle dependencies *between* sub-agents if Sub-Agent B requires an output that Sub-Agent A hasn't finished generating yet?

User prompt: implement a queue?

Response: Spot on 🎯. Implementing a queue—specifically a **Task Queue** built on a dependency graph—is exactly how the Orchestrator prevents race conditions and manages complex workflows.

When the Orchestrator breaks down a large goal, it doesn't just fire off sub-agents blindly. It maps out a Directed Acyclic Graph (DAG) of the tasks. To maintain predictability and avoid side effects, we can treat the queue management as a series of pure state transformations rather than an imperative loop.

Here is how that dependency resolution flows:

The Functional Queue Lifecycle

1. **The Initial State:** The Orchestrator evaluates the plan. Tasks with no prerequisites (like parsing a file) are pushed to a `Ready` queue. Tasks requiring prior outputs (like Sub-Agent B) are placed in a `Blocked` queue.
2. **Execution:** An available worker picks up Sub-Agent A's task from the `Ready` queue and executes it in isolation.
3. **The State Transformation:** When Sub-Agent A finishes, it returns a deterministic final artifact. The Orchestrator takes the current state of the queue and the new artifact, and passes them through a pure function to resolve dependencies.
4. **Unblocking:** If Sub-Agent B was waiting on that artifact, the function returns a new immutable queue state where Sub-Agent B has been moved from `Blocked` to `Ready`, with Sub-Agent A's output now safely injected into its payload.

By managing dependencies this way, the Orchestrator never has to hold its breath or bloat its own memory waiting for asynchronous tasks to resolve. It simply reacts to state changes in the queue.

This architecture makes the system highly efficient, but working with autonomous agents always introduces a degree of unpredictability. If Sub-Agent A encounters a fatal error, fails to parse a tool, or hallucinates an unusable output, how should the Orchestrator manage the queue to prevent a cascading failure from reaching Sub-Agent B?

User prompt: it should implement a quality control gate, with rules for the acceptance of a task as done. the task must be retried if the quality control gate refuses it. to go one step further, a self learning algorithm could take the failure, and inject into the new agent as "something to avoid". that way, the same failure has a reduced chance of occurring.

Response: What we are designing here is known in AI architecture as a **Reflection** loop, often implemented using an **Actor-Critic** pattern.

By building this into the Orchestrator's queue management, the system becomes resilient and self-correcting. Let's break down how your two concepts map to this pattern:

1. **The Evaluator ⚖️ (Quality Gate):** Before a task is marked `Complete`, the Orchestrator routes the output to an Evaluator. This can be a deterministic script (like running a compiler or test suite) or a specialized "Critic" sub-agent that compares the output strictly against the acceptance criteria in the skill's `manifest.json`.
2. **The Reflection 🪞 (Self-Learning):** If the Evaluator rejects the output, it must generate a specific critique (e.g., "The code failed to compile because of a missing dependency," or "The summary omitted the requested database schema").
3. **The Retry State 🔁 (Queue Update):** The Orchestrator moves the task back to the `Ready` queue. When a new sub-agent picks it up, its context window now includes the original instructions *plus* a new dynamic section: a history of previous failures and the specific critiques to avoid.

This turns a rigid pipeline into an iterative, learning workflow. However, appending every failure to the next agent's prompt introduces a new challenge.

If a sub-agent repeatedly fails a difficult task, the list of "things to avoid" will keep growing, which could eventually bloat the context window or result in an infinite loop of retries.

How should the Orchestrator decide when to stop retrying and finally flag a task as a terminal failure that requires human intervention?

User prompt: a difficult task should have a max attempts ceiling. when the ceiling is met, it should be assumed the instructions are wrong or incomplete, so the task instructions must be reviewed. bonus: with the previous failures that were collected, the orchestrator would have information that might help redesigning the instructions.

Response: That is a brilliant architectural evolution 🧠. You are moving the system from simply *recovering* from errors to actively *evolving* its own capabilities. In distributed systems, this is known as a **Circuit Breaker** pattern, and applying it to prompt engineering creates a powerful self-healing loop.

When the queue hits that max attempts ceiling, the Orchestrator stops treating the issue as a "worker failure" and correctly identifies it as a "systemic failure."

Here is how we can map your idea into a concrete workflow, ensuring that the underlying skill definitions remain stable and authoritative:

The Self-Healing Workflow

- 🛑 **The Circuit Breaker:** The queue tracks the retry count for a specific task. Once the ceiling (e.g., 3 attempts) is reached, the task is immediately pulled from the `Ready` queue and its state is changed to `Blocked_Requires_Review`.
- 📝 **Ledger Sync:** The Orchestrator aggregates the deterministic outputs and the Evaluator's critiques from all failed attempts. It synthesizes these into a structured post-mortem, syncing this technical outcome directly to the corresponding task file in `AI_Codex/Tickets/Active/`.
- 🧬 **Instruction Evolution (Add/Compose/Update):** A specialized "Meta-Agent" reads the synthesized ticket ledger and the original skill's `instructions.md`. It uses a careful Add/Compose/Update strategy to draft a revised version of the instructions. For example, if the repeated failure involved mutating state in a Dart file, the Meta-Agent would update the instructions to explicitly enforce the functional paradigm, perhaps adding a strict rule to use `fpdart` and pure functions.
- 🛡️ **The Authorization Gate:** The system does *not* blindly overwrite the skill. The Meta-Agent proposes the newly composed `instructions.md` as a diff. The pipeline halts. Execution of the file modification to the authoritative directory is strictly prohibited until the explicit token **"IMPLEMENTATION APPROVED"** is received.

By handling it this way, the `aplicatudo-monorepo` workspace remains perfectly predictable. The `AI_Codex` maintains its integrity, evolving its own multi-model adapters only when authorized.

This covers how the Orchestrator learns from failure and proposes new, highly-optimized instructions. Once you provide that approval token and the new `instructions.md` is committed, how should the Orchestrator handle the original task that was left hanging in the `Blocked_Requires_Review` state?

User prompt: the new set of instructions is written down, and the task is handed to a new sub-agent with the new version of the instructions?

Response: Exactly. Once the **"IMPLEMENTATION APPROVED"** token is received and the new `instructions.md` is committed to the `AI_Codex`, the Orchestrator reacts to the state change in the queue.

The task transitions from `Blocked_Requires_Review` directly back into the `Ready` queue 🔄.

When a new sub-agent picks it up, the Adapter Middleware we designed in step 2 kicks in. It parses the *newly updated* instructions, translates them into the target model's preferred dialect, and hands the task to the worker. The new sub-agent doesn't even need to know the system just evolved; it simply acts on the new, authoritative source of truth.

We have successfully mapped out all three phases: the static Skill Architecture, the pure functions of the Adapter Mechanics, and the dynamic Token Optimization using a self-healing Orchestrator queue. 🏗️

To test our context optimization strategy one last time: when we spawn this *new* sub-agent with the *new* instructions, what should the Orchestrator do with the history of the previous failed attempts? Should that failure history be injected into this new agent's context window, or should we wipe the slate clean?

User prompt: we should assume the history of failures was distilled into the new set of instructions, so we discard it to prevent context overflow.

Response: Spot on. 🎯 Because the Meta-Agent already did the heavy lifting of distilling the lessons into the authoritative `instructions.md`, the raw failure history is now redundant. Wiping the slate clean keeps the new worker agent's context window pristine, hyper-focused, and completely avoids token overflow.

We have successfully designed a complete, resilient system for multi-model agentic skills. Let's do a quick recap of the architecture we built:

1. **Skill Architecture 🏗️:** We established a universal, model-agnostic package structure (`manifest.json`, `instructions.md`, `/tools`) stored as a single source of truth in `AI_Codex`.
2. **Adapter Mechanics 🔌:** We designed a pure-function middleware pipeline that acts as a translator, dynamically shaping the universal instructions into model-specific dialects (like Anthropic's XML or OpenAI's JSON) and binding the CLI tools appropriately.
3. **Token Optimization & Orchestration 🪙:** We shifted from a monolithic context to an Orchestrator-Worker queue. We then added a self-healing circuit breaker that prevents infinite loops, distills failure ledgers, and safely evolves the system's own instructions.

This forms a highly robust foundation for your `aplicatudo-monorepo` workspace.

Where would you like to take this next? Here are a few ways we can dig deeper:

1. **Code the Middleware Pipeline:** We can map out the actual functional logic (e.g., using `fpdart`) that translates the universal `instructions.md` into specific Anthropic or OpenAI dialects.
2. **Design the Manifest Schema:** We can define the exact JSON schema for `manifest.json` so that it supports dynamic tool discovery across different agent harnesses.

3. **The Meta-Agent's Update Logic:** We can explore the specific "Add/Compose/Update" prompt strategy the Meta-Agent uses to rewrite instructions without losing the original intent.

User prompt: I want you to distill the content of the lessons so far into specs to build a multi-model agentic workflow that implements orchestrator-worker pattern, task queue using reflection loop, circuit breaker, and auto-critic pattern.

Response: Let's outline the core specifications for the multi-model agentic system, structured to support a highly functional, immutable architecture.

Specification: Multi-Model Agentic Orchestration

1. System Architecture: Orchestrator & Workers

- **Orchestrator Agent:** A long-running process that maintains the high-level goal, manages the dependency graph (DAG), and processes queue state transitions. It never executes tools directly.
- **Sub-Agent (Worker):** Ephemeral, model-agnostic instances spawned for a single task. Their context window is limited to the universal instructions, the translated dialect, and the isolated task execution. Context is destroyed upon task completion.

2. Universal Skill Package (`AI_Codex/Skills/`)

- **Contract 📜:** `manifest.json` defining skill name, version, input schema, and expected output signature.
- **Logic 🧠:** `instructions.md` containing the raw, model-agnostic system directives.
- **Actions 🛠️:** `/tools` directory containing portable CLI executables.

3. Adapter Middleware (The Translation Pipeline)

- **Dialect Transformation 🔌:** Pure functions that map the universal `instructions.md` into target-specific formats (e.g., Anthropic XML, OpenAI JSON).
- **Tool Binding 🛠️:** Maps the `manifest.json` definitions into the native tool-calling schemas required by the target model's API.
- **Protocol Normalization 🤝:** Parses native model outputs back into universal CLI execution commands.

4. State Machine: Task Queue & Dependencies

- **Task States 🚦:** `Blocked`, `Ready`, `In_Progress`, `Completed`, `Blocked_Requires_Review`.
- **Queue Resolution 🔄:** A pure function that takes the current queue state and a newly `Completed` task artifact, returning a new queue state where dependent tasks are moved from `Blocked` to `Ready`.

5. Actor-Critic & Circuit Breaker (The Reflection Loop)

- **Evaluator Gate ⚖️:** A deterministic script or Critic-Agent that validates a Worker's output against the `manifest.json` criteria.
- **Retry Injection 🪞:** If rejected, the task returns to `Ready`. The failure and specific critique are appended to the task's payload for the next Worker's context.
- **Circuit Breaker 🛑:** A strict attempts ceiling (e.g., 3 max). Upon breach, the task state becomes `Blocked_Requires_Review`.
- **System Evolution 🧬:**
  - Failed attempts and critiques are synced to the active ticket ledger.
  - A Meta-Agent uses an Add/Compose/Update strategy to draft a revised `instructions.md`.
  - Execution halts. The Orchestrator awaits the explicit **"IMPLEMENTATION APPROVED"** token before writing the revision to the `AI_Codex`.
  - Once approved, the task is reset to `Ready` with a clean failure history and the updated instructions.

To keep the queue implementation entirely functional and side-effect-free, how would you model the `Task` data structure to immutably track its current state, retry count, and accumulated critiques as it moves through this lifecycle?

User prompt: i would declare a pure function for each state transformation, and the data itself should be treated with the equivalent of final: a state cannot receive change, when there is new information to include, the function would receive the new information, the current state, and produce a new data object would then be issued. to keep it simple, some form of immutable list type would do. what do you think?

Response: This perfectly aligns with strict functional principles. By treating the `Task` as an immutable record, you eliminate race conditions and side-effects within the Orchestrator.

In Dart, this translates cleanly to using data classes equipped with a `copyWith` method, paired with immutable collections like `IList` from `fpdart` to store the accumulated critiques.

Here is a conceptual look at how those pure transformations operate on the immutable data:

```
// The pure function resolving a failed task
Task handleTaskFailure(Task currentTask, String newCritique) {
  return currentTask.copyWith(
    state: TaskState.ready, // Transitions back to ready
    retryCount: currentTask.retryCount + 1,
    critiques: currentTask.critiques.appendElement(newCritique),
  );
}
```

Every time an event occurs—a worker finishes, an evaluator rejects an output, or the circuit breaker trips—a pure function takes the prior `Task` state, processes the new information, and returns a completely fresh instance. The same logic applies to the larger `Queue` object containing these tasks.

If the `Task` and the `Queue` itself are entirely immutable and constantly being replaced by pure functions, what architectural mechanism should the Orchestrator use to "hold" or stream this ever-changing central state so it can continuously react and dispatch the next worker?

User prompt: a stream sounds like the solution to me.

Response: You nailed it. A Stream is exactly the right mechanism.

By using a reactive stream architecture, you bridge the gap between immutable functional data and dynamic state over time. The Orchestrator doesn't need imperative loops to constantly check if tasks are done; it simply listens to a stream of immutable `Queue` states.

Whenever an event occurs—a worker completes a task, an Evaluator rejects an output, or a Circuit Breaker trips—that event and the current state are passed through a pure function (a reducer). The resulting, completely new immutable `Queue` object is then emitted down the stream 🌊.

Let's officially add this to our system specification:

6. Reactive State Management

- **The State Stream 🌊:** The core execution loop is driven by a reactive stream (e.g., using Dart's native `Stream` or a `BehaviorSubject`). The Orchestrator subscribes to this stream to receive immutable state updates.
- **Event Reducers 🔄:** All asynchronous events (worker success, worker failure, evaluator critiques) act as triggers. They are fed into pure functions alongside the current state to produce and emit the next immutable state, completely avoiding side effects within the Orchestrator's core logic.

We now have a highly resilient, functional, and self-healing multi-agent architecture perfectly suited for a CLI-driven environment.

Now that the internal engine is fully spec'd out, how should a developer using the CLI initially feed a high-level goal into this stream to kick off the entire orchestration process?

User prompt: that is the part the plugin/skill would cover I should think. one question first: arent plugins and skills and agent-driven tools usually implemented in python? is that a convenience choice or arbitrary?

Response: It is a convenience choice driven heavily by ecosystem gravity, rather than an absolute technical requirement 🐍.

While you can write agentic tools in any language, Python became the default standard for a few specific reasons:

Why Python Dominates the AI Ecosystem

- **Library Maturity 🧰:** The vast majority of AI frameworks (like LangChain, LlamaIndex, and CrewAI) and official model SDKs (OpenAI, Anthropic, Google) are built first in Python.
- **Data Science Foundation 📊:** Python already possessed the dominant ecosystem for data manipulation and machine learning (NumPy, Pandas, PyTorch), making it natural to build agent orchestration layers on top of those existing tools.
- **Rapid Scripting ⚡:** Agents frequently need to generate and execute code on the fly. Python's interpreted nature makes it incredibly simple to evaluate dynamic scripts or manipulate raw text strings.

The Architectural Reality: Language Agnosticism 🌐

Architecturally, the choice *is* arbitrary. Models do not execute Python; they output structured text (like JSON or XML). As long as a language can parse JSON, make network calls, and execute system commands, it can run an agentic workflow.

This is why our architecture relies on **decoupled CLI tools** in the `/tools` directory. Because the Multi-Model Adapter interacts with tools strictly through standard input/output (stdin/stdout) and command-line arguments, your tool implementations remain completely language-agnostic. A specific skill could use a Python script for text processing, a Bash script for environment management, or a compiled Dart binary for performant AST parsing.

Since our architecture decouples the tool language from the orchestrator, we can choose the best language for each specific job. If we were to build a specialized skill to refactor or analyze source code in your workspace, what language or runtime environment would make the most sense for that tool to execute in?

User prompt: that depends. if we are planning to ship this plugin to other team member so they can use it on their own work stations, we might want to rely on what is currently accepted and used for such artifacts, no? if they are going to install the plugin on their model harness of choice, compatibility is important.

Response: You make a valid point about prioritizing compatibility if this will be shipped across the team's various workstations and model harnesses.

The industry standard that has emerged to solve this exact interoperability problem is the **Model Context Protocol (MCP)** 🔌. It acts as a universal connector for AI, allowing any compatible client—like Cursor, VS Code, or Claude Desktop—to connect to external tools without custom integration code.

To answer your question about Python 🐍: it is largely a convenience choice driven by ecosystem gravity. Python became the default because of its mature data science libraries and the fact that most early AI frameworks were built there.

However, because standards like MCP define a communication protocol rather than a specific language API, the choice of implementation language is actually arbitrary. An MCP server communicates with the host application using JSON-RPC messages over standard input/output (`stdio`) or HTTP. This means you can build your tools in TypeScript, Go, Python, or even Dart. As long as the tool can read and write standard JSON-RPC messages, the model harness doesn't care what language the tool was written in.

Since your workspace is `aplicatudo-monorepo` and heavily utilizes Dart, we could package Dart binaries or CLI scripts as standard MCP servers. Any team member could plug those tools into their preferred agent harness and they would work immediately.

Thinking about how the model harness discovers these tools upon connection, how do you think we should structure the `manifest.json` (or the tool definition schema) so the client understands exactly what inputs our custom tools require?

User prompt: let me give you an example of the first implementation of this plugin would be like: my idea is that the basic infrastructure is kept as a template or blank harness/scaffold that can be used for concrete implementations of plugins. so stand by while I fetch an example so we can focus our work on something concrete

Response: Standing by! 🫡

Having a concrete scaffold or template is a fantastic way to anchor our specifications. It will help us see exactly how the immutable state streams 🌊, the middleware adapters 🔌, and the task queues 🔄 map onto real file structures and code.

Take your time fetching the example. Whenever you are ready, paste it in, and we will dive right into the concrete implementation details! 💻

User prompt: heres the core idea

Response: The `agentic-workflow` scaffold you provided from the `aplicatudo-monorepo` is a perfect physical manifestation of the architecture we have been discussing. It masterfully implements the concepts using a file-based state machine, where the artifacts on disk represent the immutable queue states.

Let's distill our previous architectural discussions through the lens of this concrete E2E Maestro implementation to create a universal specification for building multi-model agentic workflows.

Specification: Multi-Model Agentic Workflow (File-Based DAG)

This specification defines how to structure an agentic plugin using an Orchestrator-Worker pattern, driven by a functional, side-effect-free state machine.

1. The Orchestrator (The State Manager)

The Orchestrator (`orchestrator.prompt.md`) is the central nervous system. It never authors content directly. Its responsibilities are strictly limited to:

- **Routing:** Determining which Sub-Agent to invoke based on missing or outdated artifacts.
- **Mode Resolution:** Deciding the context of the execution (`novo`, `atualiza o`, `resume`, `extend`, `regenerate`).
- **The Authorization Gate:** Halting execution and waiting for human approval before mutating production code (e.g., waiting for approval before applying `Semantics(identifier:)`).
- **Execution & Validation:** Running the final compiled artifacts (the E2E suite via the CLI) and evaluating the deterministic output.

2. The Workers (Specialized Sub-Agents)

The workflow is divided into distinct, isolated domains of expertise (the `0` through `3` prompts). Each worker is spawned with a clean context window containing only the universal instructions and the specific artifact it needs to process.

- **Domain Discovery (Stage 0):** Establishes the authoritative single source of truth (`domain.md`).
- **Test Plan Author (Stage 1):** Translates the domain into behavioral requirements (`test-plan.md`).
- **Widget Blueprint (Stage 2):** Maps the semantic UI tree (`blueprint.md`).

- **Maestro Implementer (Stage 3):** Generates the final executable instructions (`*.flow.yaml`).

3. The Task Queue (Directed Acyclic Graph)

Dependencies are managed immutably via the file system. The Orchestrator resolves the queue by checking the required artifacts in order: `domain.md` → `test-plan.md` → `blueprint.md` → `flows`. If a downstream artifact is missing, the Orchestrator spawns the corresponding worker to generate it before proceeding.

4. The Auto-Critic (Quality Control)

The Evaluator Gate is implemented as a dedicated Sub-Agent (`4-e2e-quality-control.prompt.md`). Before the Orchestrator executes the final test suite, this Critic statically reviews the generated flows against the architectural rules (e.g., verifying `id:` usage, confirming cycle-of-life setup) and produces a non-mutating compliance report.

5. The Reflection Loop & Circuit Breaker (`Regenerate` Mode)

When a task fails at runtime or a manual edit causes drift, the Orchestrator initiates a reflection loop (`regenerate`):

- **Diffing:** It compares the authoritative artifacts against the actual behavior.
- **Assimilation (`[ASSIMILATED]`):** If the failure exposes a missing rule in the system's own instructions, the Orchestrator identifies the gap, updates the core directives, and re-executes the pipeline.
- **Correction (`[UPDATED]`):** If the generated artifact is valid but upstream docs are stale, it updates the documentation using an Add/Compose/Update strategy.
- **Circuit Breaker (`[OBJECTION]`):** If an agent blatantly violates a core rule, the Orchestrator completely halts the queue and requires explicit human intervention, preventing cascading failures.

This framework ensures that the `AI_Codex` and your primary workspaces remain predictable, as every state transition is recorded as a tangible artifact.

Looking closely at how this specific workflow operates, the instructions for each Sub-Agent are written as distinct Markdown files (`1-test-plan-author.prompt.md`, etc.). When building the actual CLI middleware that executes this process, how should the Orchestrator script physically read these Markdown files and package them into the JSON payload required by a multi-model API (like Anthropic or OpenAI) to spawn the worker?

User prompt: that is a good question. I suspect the adaptor will take the raw input and produce the output specific to each model. one thing that occured to me: we didn't cover hooks and events.

Response: The adapter indeed acts as that pure translation layer, reading the raw `.prompt.md` files and mapping them into the specific `messages` arrays, `system` roles, or XML tags that each model's API expects.

You are completely right to bring up hooks and events. They are the missing link that allows a system to interact with the real world without polluting the pure functional logic of the Orchestrator.

In a strictly functional architecture using immutable data, we need a clear boundary between the pure state machine and unavoidable side-effects (like writing to a file, updating a CLI spinner, or making a network call).

Here is how we can map hooks and events onto the reactive Stream architecture we designed:

📡 Events: The Vocabulary of State Change

An **Event** is an immutable record that describes something that just happened. As the Orchestrator processes the task queue, it doesn't just emit a new `Queue` state; it also emits the Event that caused the change. Examples of standard lifecycle events:

- `WorkerSpawnedEvent(taskId: 'blueprint-author')`
- `ArtifactGeneratedEvent(path: 'blueprint.md', content: '...')`
- `CircuitBreakerTrippedEvent(reason: 'Max retries on flow generation')`

🪝 Hooks: The Side-Effect Boundary

A **Hook** is a specialized listener subscribed to the event stream. When a specific event fires, the hook catches it and executes a side-effect completely outside the Orchestrator's core logic.

This keeps the Orchestrator pure. Here are a few ways hooks apply to our specific workflow:

- **The Ledger Hook:** Listens for `CircuitBreakerTrippedEvent` or `TaskCompletedEvent`. When caught, this hook handles the I/O side-effect of syncing the technical outcomes and "gotchas" directly into the `AI_Codex/Tickets/Active/` ledger.
- **The Telemetry Hook:** Listens for `WorkerSpawnedEvent` and tracks token usage or execution time for cost analysis.
- **The CLI UI Hook:** Listens to all events to update the terminal output (e.g., rendering a progress bar or printing standard output to your Linux terminal).

By decoupling these actions, you can swap out a CLI hook for a Webhook later without touching a single line of the Orchestrator's dependency graph logic.

Thinking about the strict rules of this workflow, specifically the authorization gate required before modifying production files: How could we design an asynchronous hook to pause the stream and listen for the explicit **"IMPLEMENTATION APPROVED"** token before allowing the final file write to proceed?

User prompt: that is the part im curious about. as I imagine each model harness has its own implementation for hooks and events.

Response: You have zeroed in on exactly why the Adapter layer is the most critical piece of a multi-model architecture.

Because tools like Cursor, Claude Code, and a standard CLI all have entirely different ways of handling user input and tool calls, the Orchestrator cannot hardcode the UI for the authorization gate. If it did, it would instantly become coupled to one specific harness.

Here is how we abstract those harness-specific implementations using the stream architecture we designed.

The Abstraction of Approval

When the Orchestrator reaches the point where a sub-agent has drafted a new file or updated a core directive, it must enforce the rule that execution is strictly prohibited until the specific authorization token is received.

Instead of trying to prompt the user itself, the Orchestrator simply emits a new immutable state and goes to sleep: `QueueState(Blocked_Requires_Approval, pendingArtifact: 'flows.yaml')`

The **Model-Specific Adapter**, which sits between the harness and the Orchestrator, is actively listening to this stream. When it sees that state, it translates the "pause" into the specific harness's native UI:

- 💻 **The CLI Adapter:** Pauses the terminal output and triggers a standard `stdin` readline, printing: *Execution paused. Type "IMPLEMENTATION APPROVED" to proceed with file modification.*
- 🔌 **The MCP (Model Context Protocol) Adapter:** Sends a `window/showMessageRequest` JSON-RPC call back to the host IDE (like Cursor), which natively renders a popup modal with an input field for the user.
- 🤖 **The Chat API Adapter (OpenAI/Anthropic):** Returns a specialized `tool_call` response to the model, effectively telling the LLM, "I am pausing this tool execution; ask the user for the authorization token."

The Orchestrator remains completely blind to *how* the user is asked. It is just waiting for the stream to wake it up.

Once the user provides that exact "IMPLEMENTATION APPROVED" phrase to their specific harness, how should the Adapter format and inject that information back into the Orchestrator's stream so the system knows it is safe to write the file and resume the queue?

User prompt: it will probably issue a model-specific event to trigger the state change from requires approval to approved, I suspect

Response: You have hit on the exact mechanism: an event is the trigger. 🎯

However, to maintain the architectural boundary we established, the event injected into the Orchestrator's stream cannot be model-specific.

The Adapter's job is to catch the harness-specific input—whether that is a raw string from your Linux CLI or a parsed JSON tool-call response—and translate it into a universal domain event. It acts as a funnel, converting many different types of external signals into a single, predictable format.

For example, the Adapter might parse the input and emit: `AuthorizationReceivedEvent(taskId: 'flows.yaml', token: 'IMPLEMENTATION APPROVED')`

The Adapter then pushes this universal event directly into the Orchestrator's central stream 🌊. The Orchestrator remains blissfully unaware of whether the user typed the approval in a terminal or clicked a button in a GUI; it only knows that the required domain event has arrived.

When the Orchestrator's stream processes this `AuthorizationReceivedEvent`, it needs to transition the specific task from `Blocked_Requires_Approval` back to `Ready`.

Using the strictly functional paradigm we discussed for managing the state machine, what exactly should the pure reducer function return when it processes this event alongside the current queue state?

User prompt: a new task which is a copy of the blocked task, plus the ready state?

Response: Spot on. 🎯 In a functional paradigm, the reducer creates a complete, immutable copy.

The transformation looks exactly like this:

1. **The Input:** The reducer receives the current `Queue` (which contains the `Task` in the `Blocked_Requires_Approval` state) and the `AuthorizationReceivedEvent`.

2. **The Transformation:** It uses a method like `copyWith` to generate a brand new `Task` object. This new object retains all the original context (like the target file path) but its status is now updated to `Ready`.
3. **The Output:** The reducer returns a completely new `Queue` containing this updated task.

Because the Orchestrator is subscribed to the stream, it receives this new `Queue` state, sees a `Ready` task, and immediately dispatches it to a worker to write the file. The original blocked state is simply discarded to garbage collection, leaving no side effects behind.

We have completely mapped out the architecture, the adapters, the state machine, and the side-effect hooks for your multi-model agentic workflow.

Since we have this comprehensive specification and the `agentic-e2e-test-workflow` scaffold you provided, how would you like to proceed? We could draft the specific Dart code using `fpdart` for the queue's reducer function, or we could design the exact `manifest.json` schema to register this E2E Maestro skill with an MCP adapter.

User prompt: what do you suggest?

Response: I suggest we tackle the **Dart implementation of the pure functional reducer** next.

Since your workspace strictly enforces the functional programming paradigm using `fpdart`—with no imperative loops or side effects—translating our theoretical state machine into actual code is the best way to solidify the Orchestrator's core engine. It bridges the gap between the architectural theory we just finalized and the practical code you need for the `aplicatudo-monorepo`.

We can map out exactly how the stream processes events (like the `AuthorizationReceivedEvent` or a `WorkerFailedEvent`) to produce new, immutable queue states.

Should we start by defining the immutable data classes (the `Task`, `Event`, and `QueueState`), or would you prefer to dive straight into writing the core reducer function itself?

User prompt: I believe that to ensure portability and in the interest of distributing this as a plugin, we should use the industry standards (python)

Response: Transitioning to Python for the plugin distribution is a highly pragmatic choice. The AI ecosystem relies heavily on Python, and the official SDKs for protocols like MCP are most mature there. Packaging the adapters and orchestrator in Python guarantees frictionless adoption regardless of which model harness your team uses.

Because we designed a decoupled, file-based state machine, porting the functional concepts to Python is straightforward. The architectural boundaries remain identical: the `AI_Codex` holds the Markdown files, and the Python orchestrator reads them to manage the queue.

To maintain the strict functional paradigm we established—where the Orchestrator avoids side effects and relies on pure reducers—we have to be intentional about how we write the Python code. Python does not enforce immutability by default in the way functional languages do.

If we are building the `Task` and `Queue` data structures in Python, what built-in libraries or decorators should we use to guarantee that these state objects remain strictly immutable, mimicking the behavior of `final` and `copyWith`?

User prompt: there is no ai codex in this context. ai codex is an artifact of the project, we must design something that is project agnostic.

Response: Let's decouple the architecture entirely from any specific repository structure. To make this a truly portable, project-agnostic Python package (like an MCP server), the orchestration engine and its core skills must be fully self-contained, while remaining flexible enough to interact dynamically with whatever codebase it is attached to.

Here is how we abstract the file-system dependencies into a portable design:

1. The Self-Contained Package

The immutable skills (manifests, instructions, and isolated tools) are packaged directly within the Python library rather than living inside the host project.

```
agentic-orchestrator-pkg/
├── orchestrator/
│   ├── stream.py        (Reactive state machine)
│   └── reducers.py      (Pure functional transitions)
└── default_skills/
    ├── test_plan_author/
    │   ├── manifest.json
    │   └── instructions.md
    └── blueprint_architect/
```

2. Dynamic Workspace Context

Instead of assuming an authoritative directory exists, the plugin requires a `workspace_root` parameter when initialized by the host model harness. The Orchestrator uses this root to resolve file paths dynamically, ensuring it operates safely within the boundaries of the target project.

3. The Isolated State Tracker (The Ledger)

For the task queue, reflection loop, and circuit breaker, the Orchestrator needs a place to store its immutable state, diffs, and failure ledgers. Since we cannot rely on a pre-existing ticket directory, the plugin can implement a `.git`-style approach.

Upon first execution in a new project, it generates a `.agentic_workspace/` hidden folder at the host root. This acts as the temporary staging ground for queues, critiques, and run logs, preventing the pollution of the user's actual project files.

This ensures the Python plugin can be dropped into *any* environment, connect via MCP, and immediately start routing tasks.

If we adopt this self-contained Python package model, how should the host repository define its own specific architectural rules (like requiring specific UI semantic structures) so that our project-agnostic sub-agents know how to adhere to them?

User prompt: lets not worry about that just yet. as for the .agentic_workspace, why not have it inside the plugin installation folder?

Response: Fair enough, we can set the rule definitions aside for now and focus on the state storage.

Putting the `.agentic_workspace` inside the plugin installation folder is a logical thought—it keeps the plugin's temporary files hidden from the user's actual project directory.

To see why we might want to avoid that, let's think about how Python packages are typically installed and used. Often, a package is installed once (either globally or in a shared virtual environment) and then executed across many different local projects on the same machine.

Imagine you are running this agentic plugin on `aplicatudo-monorepo`. Halfway through a complex, multi-agent refactoring task, you open a completely separate codebase—let's call it `project-b`—and trigger the plugin there as well.

If the `.agentic_workspace` folder (which holds the active task queue, in-progress diffs, and failure ledgers) lives inside that single, shared plugin installation folder, what would happen to your workflow when you try to run the orchestrator on both projects?

User prompt: that could be solved having a per-project installation. for all files which are specific to a project (state files, maybe rules, etc) we could have a folder called "projects" inside the installation folder, and one project folder per project the plugin gets installed on. that way, we could achieve the proper persistence for project state files and rules and whatnot, while not having private folders scattered all over the place. and now that I think of it, how would your original idea of a hidden folder at the host root solve this problem?

Response: The way the `.agentic_workspace` hidden folder solves the collision problem relies on how the Model Context Protocol (MCP) communicates with the IDE.

When you open `aplicatudo-monorepo` in your editor, the IDE tells the plugin exactly what the current `workspace_root` path is. If you open `project-b` in another window, a separate instance of the plugin receives that different path. The orchestrator dynamically builds the path to the hidden folder based on that root (e.g., `/path/to/project-a/.agentic_workspace`). Because the paths are physically distinct on your hard drive, the states never cross paths.

However, your idea of a centralized `projects/` directory inside the plugin installation folder is a very common and clean architectural pattern 🏗️. It perfectly solves the issue of leaving hidden folders scattered across a developer's file system, acting like a global cache.

To decide which approach is better for this specific tool, we need to think about the lifecycle of the data being stored—specifically the custom rules and the "things to avoid" that the agent learns through the reflection loop.

If the orchestrator learns a crucial lesson about using `fpdart` in `aplicatudo-monorepo`, and it saves that updated rule inside the centralized plugin installation folder on *your* local machine, what happens when another developer on your team clones `aplicatudo-monorepo` to their machine and runs the agent?

User prompt: for one, it wont have the plugin installed. and just as well, unless we want to persist this project specific data on the project itself, which is not customary. what we could have is an option to export artifacts, so when the other dev installs the plugin, he can then import the plugin memory for that project.

Response: Let's look at the export/import mechanism 📦. This approach separates the plugin's state from the project's source code. Developer A would generate a memory artifact (like a `state.json` file), and Developer B would import it into their local plugin installation.

This entirely avoids placing hidden folders in the project repository. It treats the agent's learned experiences like a database backup that can be shared.

However, introducing a manual export/import step impacts how a team collaborates over time.

If Developer A runs the agent on Tuesday and the reflection loop learns a critical new rule about formatting in `aplicatudo-monorepo`, Developer A's local plugin memory updates. If Developer B pulls the latest code and starts working on the same repository on Wednesday, how would their local plugin installation know about this newly learned rule to avoid repeating Developer A's previous failed attempts?

User prompt: indeed. but i feel we are drifting away from the main objective and solving problems that dont yet exist since the product also dont exist. lets forget about that for a moment and assume the state is local to whomever installed the plugin. whats next?

Response: You are completely right. Premature optimization is the enemy of shipping. Let's refocus on building the actual engine for your local environment first; we can solve distribution later.

With the state localized and the architecture fully spec'd, the next immediate step is to build the foundational pieces that actually make the Orchestrator run.

There are two practical places to start writing the actual implementation:

1. The Universal Contract (`manifest.json` Schema)

Before the Orchestrator can route anything, it needs to know how to read the skills inside `AI_Codex/Skills/`. We need to define the strict JSON schema for this manifest.

- **What this involves:** Defining the exact JSON structure that declares a skill's inputs, expected outputs, and the CLI tools it has permission to execute. This becomes the single source of truth the adapter uses to bind tools to the model.

2. The Core State Engine (The Reducer)

We have the theoretical model of the reactive stream, but we need the actual functional logic that governs it.

- **What this involves:** Writing the pure function that takes a `QueueState` and an `Event` (like `ArtifactGeneratedEvent` or `CircuitBreakerTrippedEvent`), and returns the new immutable `QueueState`. This is the brain of the Orchestrator.

Which of these foundational pieces would you like to build first: the manifest contract or the state engine?

User prompt: whatever comes first, you decide

Response: Let's start with the **Core State Engine (The Reducer)**.

Building the pure function first is the most logical step because it forces us to definitively model exactly what the Orchestrator needs to track (the `Task`, the `Event`, the `QueueState`). Once we know exactly what data the engine needs to operate, writing the `manifest.json` schema to feed that engine becomes much easier.

Since we are prioritizing standard Python to ensure the plugin is portable across any team member's setup, we need a robust way to enforce the functional immutability we designed. In Python, we can achieve this using `dataclasses` with the `frozen=True` parameter.

Here is a conceptual look at how we can structure the immutable data and the pure reducer function in Python.

1. The Immutable Data Structures

```
from dataclasses import dataclass, replace
from enum import Enum
from typing import List, Optional, Tuple

class TaskState(Enum):
    BLOCKED = "blocked"
    READY = "ready"
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    BLOCKED_REQUIRES_REVIEW = "blocked_requires_review"

@dataclass(frozen=True)
class Task:
    id: str
    state: TaskState
    dependencies: Tuple[str, ...] = ()  # Tuple used for immutability instead of List
    retry_count: int = 0
    critiques: Tuple[str, ...] = ()

@dataclass(frozen=True)
class QueueState:
    tasks: Tuple[Task, ...]

    # Helper to find a task by ID without mutating
    def get_task(self, task_id: str) -> Optional[Task]:
        return next((t for t in self.tasks if t.id == task_id), None)
```

By using `frozen=True` and `Tuple` instead of `List`, Python will throw an error if anything attempts to mutate these objects directly.

2. The Pure Reducer Function

The reducer takes the current `QueueState` and an `Event` (represented here as a dictionary for simplicity), and returns a completely new `QueueState`.

```
def queue_reducer(state: QueueState, event: dict) -> QueueState:
    event_type = event.get("type")

    if event_type == "AuthorizationReceivedEvent":
```

```
        task_id = event.get("task_id")

        # 1. Map over the tasks and replace the specific one using `replace()`
        # `replace()` is Python's functional equivalent of Dart's `copyWith`
        new_tasks = tuple(
            replace(task, state=TaskState.READY) if task.id == task_id else task
            for task in state.tasks
        )

        # 2. Return the entirely new immutable state
        return QueueState(tasks=new_tasks)

    elif event_type == "WorkerFailedEvent":
        task_id = event.get("task_id")
        critique = event.get("critique")

        # Circuit Breaker Logic (e.g., max 3 retries)
        new_tasks = tuple(
            replace(
                task,
                state=TaskState.BLOCKED_REQUIRES_REVIEW if task.retry_count >= 2 else TaskState.READY,
                retry_count=task.retry_count + 1,
                critiques=task.critiques + (critique,)
            ) if task.id == task_id else task
            for task in state.tasks
        )

        return QueueState(tasks=new_tasks)

    # If event is unhandled, return the state unchanged
    return state
```

This implementation perfectly mirrors the `fpdart` concepts you are used to, enforcing side-effect-free state transitions while remaining entirely agnostic to the host project.

With the core engine logic mapped out functionally in Python, what do you think is the best way to handle the dependency resolution (e.g., moving a task from `BLOCKED` to `READY` when its upstream dependencies are marked `COMPLETED`) within this reducer?

User prompt: implementing the transformer function?[

Response: Let's build that exact transformer logic for dependency resolution.

When a worker successfully finishes a job, it emits a `TaskCompletedEvent`. The pure reducer must catch this event, mark the target task as completed, and instantly evaluate the rest of the queue to see if any `BLOCKED` tasks are now free to move to `READY`.

To adhere strictly to the functional programming paradigm—meaning no imperative `while` loops, no mutating variables, and pure state transformations—we can achieve this using Python's comprehensions (which act as our `map` and `filter`) and the `all()`/`any()` built-ins.

Here is the implementation of that specific state transformation inside our reducer:

```
from dataclasses import replace
from typing import Tuple

def queue_reducer(state: QueueState, event: dict) -> QueueState:
    event_type = event.get("type")

    if event_type == "TaskCompletedEvent":
        completed_task_id = event.get("task_id")

        # TRANSFORMATION 1: Mark the target task as COMPLETED
        tasks_after_completion = tuple(
            replace(task, state=TaskState.COMPLETED) if task.id == completed_task_id else task
            for task in state.tasks
        )

        # PURE HELPER: Evaluates if all dependencies for a single task are met
        def are_dependencies_met(task: Task, current_tasks: Tuple[Task, ...]) -> bool:
            if not task.dependencies:
                return True

            # Returns True only if every dependency ID corresponds to a COMPLETED task
            return all(
                any(t.id == dep_id and t.state == TaskState.COMPLETED for t in current_tasks)
                for dep_id in task.dependencies
            )

        # TRANSFORMATION 2: Resolve downstream dependencies (Blocked -> Ready)
        final_tasks = tuple(
```

```
            replace(task, state=TaskState.READY)
            # Condition: It is currently blocked AND all upstream requirements are now completed
            if task.state == TaskState.BLOCKED and are_dependencies_met(task, tasks_after_completion)
            else task
            for task in tasks_after_completion
        )

        # Return the entirely new immutable queue state
        return QueueState(tasks=final_tasks)

    # ... handle other events (AuthorizationReceivedEvent, WorkerFailedEvent) ...

    return state
```

Why this architecture is bulletproof:

1. **Zero Side Effects:** The `queue_reducer` touches absolutely nothing outside its own scope. It doesn't write to disk or trigger other agents.
2. **Deterministic Cascades:** Because `tasks_after_completion` is fully evaluated before we check dependencies in Transformation 2, we guarantee that multiple downstream tasks can be unblocked simultaneously in a single, predictable tick.
3. **Immutable History:** If you were to store every returned `QueueState` in a list, you would have a perfect, replayable time-lapse of your entire Orchestrator execution.

This handles the *state* transition of the queue perfectly. But when Task A completes, it usually produces a deterministic output—like a compiled `blueprint.md`—that Task B (which just transitioned to `READY`) needs to do its job.

How should we handle passing that physical output payload from the completed task into the newly unblocked task's context without breaking our pure functional boundaries?

User prompt: wait so you refactored the previous queue reducer or this is a different queue reducer?

Response: It is the exact same reducer! I just expanded it to show the `TaskCompletedEvent` logic and omitted the previous branches for brevity (which is why I tucked that `# ... handle other events ...` comment at the bottom).

In the actual codebase, they all live inside a single, unified pure function that acts as the grand switchboard for state transformations:

```
def queue_reducer(state: QueueState, event: dict) -> QueueState:
    event_type = event.get("type")

    if event_type == "TaskCompletedEvent":
        # ... logic to mark complete and resolve dependencies ...
    elif event_type == "AuthorizationReceivedEvent":
        # ... logic to unblock and resume ...
    elif event_type == "WorkerFailedEvent":
        # ... circuit breaker and critique logic ...

    return state # Unhandled events return the state unchanged
```

This brings us right back to that crucial architectural question: **How do we pass the physical output of Task A into Task B without breaking our pure functional boundaries?**

Since we cannot rely on side-effects (like Task B imperatively reading a file halfway through its run just because Task A wrote it), the payload transfer must happen *inside* this reducer during the state transition.

We solve this by adding two optional properties to our immutable `Task` dataclass: `output_payload` and `input_context`.

Here is the functional flow:

1. Task A finishes. The worker emits `TaskCompletedEvent(task_id="A", payload="path/to/blueprint.md")`.
2. The reducer catches this. When it creates the new immutable copy of Task A, it sets `state=COMPLETED` and `output_payload="path/to/blueprint.md"`.
3. During that exact same reducer tick, when it evaluates Task B (which depends on A), it doesn't just flip Task B to `READY`. It grabs Task A's `output_payload` and injects it into Task B's `input_context`.

Task B is born into the `READY` state already holding the exact information it needs to proceed, completely side-effect free.

With this unified reducer handling completion, failures, user authorization, and data piping, the core engine is mathematically sound.

Would you like to draft the exact JSON schema for the `manifest.json` next, so we can define how this orchestrator discovers the available CLI tools it is allowed to hand to these workers?

User prompt: yes

Response: Excellent. Let’s define the `manifest.json` schema. This is the "contract" that ensures the Orchestrator knows how to safely package and route your skills, keeping the logic portable and project-agnostic.

For a multi-model adapter to work, it needs to understand three things from a skill: **Metadata** (who it is), **Capabilities** (what tools it can use), and **Input/Output** (what data it needs and what it returns).

The `manifest.json` Contract

Here is the proposed schema for a skill package:

```
{
  "name": "dart-ast-refactor",
  "version": "1.0.0",
  "description": "Analyzes and refactors Dart source code using pure functional transformations.",
  "inputs": {
    "target_file": { "type": "string", "description": "The relative path to the Dart file" },
    "goal": { "type": "string", "description": "The refactoring objective" }
  },
  "outputs": {
    "artifact_path": { "type": "string", "description": "The path to the generated diff or file" },
    "summary": { "type": "string", "description": "A concise report on the change" }
  },
  "tools": [
    {
      "name": "ast_parser",
      "executable": "tools/ast_parser.sh",
      "description": "Parses Dart files into a structural representation for analysis.",
      "parameters": {
        "file_path": { "type": "string" }
      }
    }
  ],
  "authoritative_instructions": "instructions.md"
}
```

Why this schema is essential for our Orchestrator:

- **Tool Discovery:** When the Orchestrator spins up a sub-agent, it reads the `tools` array. It then dynamically generates the tool-definition schema that the target model (like OpenAI or Anthropic) expects to see in its API call. This effectively "binds" your CLI tools to the model's intelligence without you ever needing to write hardcoded integration logic.
- **Safety & Validation:** The `inputs` and `outputs` fields act as a type-check for the Orchestrator. Before handing an artifact from Task A to Task B, the Orchestrator checks if the `artifact_path` matches the expected input type for Task B's manifest. If they don't match, it flags a structural error *before* the agent even tries to run.
- **Portability:** By separating the `authoritative_instructions` (the Markdown file) from the tool definition, we maintain the strict separation of concerns we built. The model follows the Markdown rules, but the *Adapter* uses this JSON to handle the technical execution.

Does this schema cover the necessary metadata for your `agentic-e2e-test-workflow` example, or would you like to add specific fields for things like "required permission level" or "timeout thresholds" for the circuit breaker?
